

Core Insight (Important)

In MT5, an EA does NOT need to be attached to a symbol chart to trade that symbol.

An EA:

- Is attached to one chart only (any symbol, any timeframe)
- Can subscribe to, monitor, and trade ANY symbol available on the broker

Your issue is architectural, not a platform blocker.

The Correct Architecture (Single-Chart, Multi-Asset EA)

One EA

- Attached to ONE chart only (e.g. EURUSD M1)
- Acts as a trade router / execution engine
- Communicates with your WebSocket → EA bridge
- Trades any symbol dynamically

Step 1: Symbol Subscription (Critical)

MT5 will not stream prices for symbols unless they're subscribed.

You must explicitly select them:

```
bool EnsureSymbol(string symbol)
{
    if(!SymbolSelect(symbol, true))
    {
        Print("Failed to select symbol: ", symbol);
        return false;
    }
    return true;
}
```

Call this before trading or reading prices.

Step 2: EA Trades Any Symbol (No Chart Needed)

You can trade any symbol from the EA like this:

```
#include <Trade/Trade.mqh>
CTrade trade;

void ExecuteTrade(string symbol, ENUM_ORDER_TYPE
type, double volume)
{
    EnsureSymbol(symbol);
    trade.SetExpertMagicNumber(123456);

    if(type == ORDER_TYPE_BUY)
        trade.Buy(volume, symbol);
    else if(type == ORDER_TYPE_SELL)
        trade.Sell(volume, symbol);
}
```

Step 3: Market Data Without Charts

You can read prices for any symbol:

```
double bid = SymbolInfoDouble(symbol, SYMBOL_BID);  
double ask = SymbolInfoDouble(symbol, SYMBOL_ASK);  
Or full tick stream:
```

```
MqlTick tick;  
SymbolInfoTick(symbol, tick);
```

Step 4: WebSocket → EA Message Model

Your EA should behave like a stateless execution engine.

Example JSON message from WebSocket:

```
{  
  "action": "trade",  
  "symbol": "XAUUSD",  
  "side": "buy",  
  "volume": 0.5,  
  "sl": 2345.0,  
  "tp": 2360.0  
}
```

EA logic:

1. Parse message
2. SymbolSelect(symbol)
3. Execute trade
4. Send execution result back

Step 5: Handling Symbol Discovery Automatically

At startup, the EA can load all broker symbols:

```
void SubscribeAllSymbols()  
{  
    int total = SymbolsTotal(true);  
    for(int i=0; i<total; i++)  
    {  
        string symbol = SymbolName(i, true);  
        SymbolSelect(symbol, true);  
    }  
}
```

Or better: lazy-load symbols only when requested (recommended for performance).

Step 6: No Need for OnTick per Symbol

Important clarification:

- OnTick() only fires for the chart symbol
- BUT trading & price queries work independently

-

Your WebSocket messages should trigger execution directly — not rely on ticks.

Best-Practice Production Setup (What You Actually Want)

EA Role

- Attached to ONE hidden chart
- No indicators
- No symbol-specific logic
- Acts as:
 - WebSocket client
 - Symbol manager
 - Trade executor
 - Position reporter

Server Role (client)

- Decides:
 - Which symbol
 - Direction

- Size
- Risk rules
- MT5 EA only executes

Common Mistakes to Avoi

- ✗** Attaching EA to every chart
- ✗** Creating multiple EA instances
- ✗** Relying on OnTick() for execution logic
- ✗** Assuming symbols must have charts

TL;DR

You only need ONE EA on ONE chart.

That EA can:

- Trade every broker symbol
- Subscribe dynamically
- Execute from WebSocket commands
- Never be manually moved again

Go never talks to MT5 directly; it talks to your EA via WebSocket.

Architecture (minimal)

- MT5 EA (MQL5)
 - exposes broker symbol list + symbol specs via WebSocket
- Go service
 - builds a canonical → broker symbol map
 - sends execution commands using the resolved broker symbol

Step 1 — EA sends broker symbol metadata (once on connect)

EA sends this JSON over WebSocket:

```
{
  "type": "symbols",
  "data": [
    { "symbol": "XAUUSD", "path": "Metals",
      "contract": 100, "description": "Gold" },
    { "symbol": "GOLD", "path": "Metals",
      "contract": 100, "description": "Gold Spot" },
    { "symbol": "US100", "path": "Indices",
      "description": "NASDAQ 100" }
  ]
}
```

Your Go service stores this.

Step 2 — Canonical symbol model (Go)

You define one internal naming standard:

```
type CanonicalSymbol string

const (
    GOLD    CanonicalSymbol = "GOLD"
    NASDAQ  CanonicalSymbol = "NASDAQ"
)
```

Step 3 — Symbol registry (Go)

```
type BrokerSymbol struct {
    Symbol      string
    Description  string
    Path        string
    Contract    float64
}

var brokerSymbols = []BrokerSymbol{}
```

Populate this when the EA sends the symbol list.

Step 4 — Automatic symbol resolution logic (Go)

```
func ResolveSymbol(c CanonicalSymbol) (string,
error) {
    switch c {
    case GOLD:
        return match([]string{"XAU", "GOLD"},
[]string{"Gold"})
    case NASDAQ:
        return match([]string{"US100", "NAS"},
[]string{"NASDAQ"})
    default:
        return "", errors.New("unknown canonical
symbol")
    }
}

func match(symbolHints, descHints []string)
(string, error) {
    for _, s := range brokerSymbols {
        for _, h := range symbolHints {
            if
strings.Contains(strings.ToUpper(s.Symbol), h) {
                return s.Symbol, nil
            }
        }
        for _, d := range descHints {
            if
strings.Contains(strings.ToUpper(s.Description),
strings.ToUpper(d)) {
                return s.Symbol, nil
            }
        }
    }
}
```

```

    return "", errors.New("no broker symbol match")
}

```

This works across:

- XAUUSD / GOLD / GOLD.a
- US100 / NAS100 / USTEC
- Broker suffixes

Step 5 — Execution command (Go → EA)

```

type TradeCommand struct {
    Action string `json:"action"`
    Symbol string `json:"symbol"`
    Side   string  `json:"side"`
    Volume float64 `json:"volume"`
}

func SendTrade(ws *websocket.Conn, c
CanonicalSymbol, side string, vol float64) error {
    symbol, err := ResolveSymbol(c)
    if err != nil {
        return err
    }

    cmd := TradeCommand{
        Action: "trade",
        Symbol: symbol,
        Side:   side,

```

```
        Volume: vol,  
    }  
  
    return ws.WriteJSON(cmd)  
}
```

Why this is the correct approach

- No hardcoded broker symbols
- No manual mapping per broker
- One EA, one WebSocket
- Broker-agnostic execution layer

Final takeaway

- Go handles logic, mapping, intelligence
- EA handles execution and broker truth
- Symbol names are resolved automatically from live broker data